

TelemetriaMqtt v2 — Checkpoint 5

Schema e setup do InfluxDB

Projeto de Telemetria FSAE

16 de setembro de 2026

Abstract

Este documento detalha o quinto checkpoint do projeto: colocar o InfluxDB no ar como container Docker, confirmar de fato a suposição de versão (2.x) feita no checkpoint 4, e validar contra um banco real – não mais mockado – a escrita e a leitura de pontos de telemetria.

Contents

1	Contexto e objetivo	1
2	Estrutura adicionada	1
2.1	Serviço no docker-compose.yml	2
2.2	Credenciais via infra/.env	2
2.3	Schema	2
3	Testes	2
3.1	Resultado da execução	3
4	Descoberta durante a validação manual	3
5	Nota sobre metodologia	3
6	Resumo do que ficou pronto	3
7	Próximos passos	4

1 Contexto e objetivo

No checkpoint 4, o script de ingestão foi escrito e testado com um `write_api` mockado (`unittest.mock.MagicMock`), porque o InfluxDB ainda não existia como serviço. A suposição de que o banco seria InfluxDB 2.x (baseada na terminologia de "bucket" usada nos slides do TCC antigo) ficou registrada no `SCOPE.md` como pendente de confirmação. Este checkpoint resolve essa pendência: coloca um InfluxDB 2.x real no ar, e valida contra ele.

2 Estrutura adicionada

Listing 1: Arquivos novos/alterados

```
1 infra/
2   docker-compose.yml      (+ serviço influxdb)
3   .env.example            (+ variáveis do InfluxDB)
```

```

4 requirements-test.txt      (+ influxdb-client)
5 tests/
6   conftest.py              (+ fixture ensure_env_file, influxdb_instance)
7   test_influxdb.py         (novo)
8   test_docker_compose.py   (+ teste do serviço influxdb)
9   test_end_to_end.py       (+ teste do pipeline completo)

```

2.1 Serviço no docker-compose.yml

Listing 2: Trecho de infra/docker-compose.yml

```

1 influxdb:
2   image: influxdb:2
3   environment:
4     DOCKER_INFLUXDB_INIT_MODE: setup
5     DOCKER_INFLUXDB_INIT_USERNAME: ${INFLUXDB_INIT_USERNAME:-admin}
6     DOCKER_INFLUXDB_INIT_PASSWORD: ${INFLUXDB_INIT_PASSWORD:?...}
7     DOCKER_INFLUXDB_INIT_ORG: ${INFLUXDB_INIT_ORG:-fsae}
8     DOCKER_INFLUXDB_INIT_BUCKET: ${INFLUXDB_INIT_BUCKET:-telemetria}
9     DOCKER_INFLUXDB_INIT_ADMIN_TOKEN: ${INFLUXDB_INIT_ADMIN_TOKEN:?...}
10  volumes:
11    - influxdb-data:/var/lib/influxdb2

```

A imagem oficial faz o setup inicial (usuário, organização, bucket, token de admin) sozinha, lendo variáveis de ambiente, na primeira subida com o volume de dados vazio.

2.2 Credenciais via infra/.env

Seguindo o mesmo padrão já usado para o `passwd` do Mosquito: nenhuma credencial real em arquivo versionado. `infra/.env` (gitignored) guarda usuário, senha, organização, bucket e token; `infra/.env.example` é o template versionado. Uma *fixture* `autouse` do `pytest` (`ensure_env_file`) garante que `.env` existe antes de qualquer teste, gerando valores aleatórios se ele não existir – e nunca sobrescrevendo um `.env` real que o desenvolvedor já tenha configurado.

2.3 Schema

- **Organização:** `fsae`. **Bucket:** `telemetria`.
- **Retenção:** infinita (sem limite configurado) – decisão consciente, já que o objetivo é comparar treinos e estudar tendências ao longo de uma temporada inteira, não só o evento mais recente.
- **Measurement:** `telemetry`, já definido no checkpoint 4 (`backend/ingestion/transform.py`) – um ponto por mensagem MQTT, os 20 sinais como campos.

3 Testes

Quatro testes cobrem este checkpoint especificamente (mais dois de checkpoints anteriores, estendidos):

Teste 1 `test_point_written_via_production_writer_is_queryable` – escreve um ponto usando o `make_influx_writer` de produção (checkpoint 4) contra um InfluxDB efêmero real, e confirma via consulta Flux que o valor volta certo.

Teste 2 `test_multiple_fields_of_same_point_are_all_queryable` – confirma que múltiplos campos do mesmo ponto (`engineTemp`, `airTemp`) são gravados e consultados corretamente juntos.

Teste 3 `test_docker_compose_up_starts_a_working_influxdb` – o `docker-compose.yml` real (não uma cópia da config) sobe um InfluxDB que responde no endpoint de saúde.

Teste 4 `test_full_pipeline_mock_publisher_to_real_influxdb` – fecha o ciclo do caminho histórico inteiro: mock publisher → broker real → ingestão → InfluxDB real → consulta de volta, nenhum elo simulado.

3.1 Resultado da execução

Listing 3: Saída real de `pytest -v` (arquivos deste checkpoint)

```
1 test_influxdb.py::test_point_written_via_production_writer_is_queryable
  PASSED
2 test_influxdb.py::test_multiple_fields_of_same_point_are_all_queryable PASSED
3 test_docker_compose.py::test_docker_compose_up_starts_a_working_influxdb
  PASSED
4 test_end_to_end.py::test_full_pipeline_mock_publisher_to_real_influxdb PASSED
```

4 Descoberta durante a validação manual

Depois de gerar um novo `infra/.env` (com um token de admin novo) e rodar `docker compose up`, uma tentativa de escrita real falhou com `401 Unauthorized`. Causa: o InfluxDB já tinha sido inicializado antes, numa sessão de teste anterior, e seu volume de dados (`influxdb-data`) persistiu entre execuções – a imagem oficial só roda o processo de setup (que lê as variáveis de ambiente) na **primeira** subida com o volume vazio. Trocar `.env` depois não tem efeito nenhum até o volume ser apagado.

A correção foi `docker compose down -v` (remove os volumes) seguido de `up` novamente, forçando uma reinicialização completa. Esse comportamento foi documentado no `SCOPE.md` e no `CLAUDE.md` como nota operacional, pra não ser redescoberto por acidente mais pra frente.

5 Nota sobre metodologia

Diferente dos checkpoints anteriores, neste a implementação (serviço no `docker-compose.yml`, credenciais) foi escrita antes dos testes automatizados – os testes vieram depois de uma validação manual já confirmar que tudo funcionava. Essa inversão de ordem foi identificada e discutida com o usuário após o checkpoint 6; a partir do checkpoint 7 o processo volta a ser teste-primeiro, inclusive para infraestrutura declarativa. A cobertura de testes deste checkpoint continua válida (roda contra infraestrutura real, não é simulada) – só a ordem em que foi escrita que não seguiu TDD estrito.

6 Resumo do que ficou pronto

- InfluxDB 2.x real, container Docker, setup automático via variáveis de ambiente.
- Credenciais nunca em arquivo versionado – mesmo padrão do Mosquitto.
- Schema definido e documentado: organização, bucket, retenção infinita, measurement.
- Suposição de design do checkpoint 4 (escrita/leitura via `influxdb-client`) validada contra banco real, não mais mockada.
- Descoberta operacional sobre o comportamento de inicialização do InfluxDB documentada.

7 Próximos passos

Checkpoint 6: dashboards Grafana usando este InfluxDB como fonte de dados para o caminho histórico da arquitetura.